

- **Introduction to Algorithm Analysis**
 - **Characteristics of Algorithms**
 - **Criteria for Analysing Algorithms**
 - **Time and Space Complexity**
 - **Best, Worst and Average Case Complexities**
 - **Asymptotic Notations**
 - **Big-Oh (O), Big- Omega (Ω), Big-Theta (Θ), Little-oh (o) and Little- Omega (ω) and their properties.**
 - **Classifying functions by their asymptotic growth rate**
 - **Time and Space Complexity Calculation of simple algorithms**
 - **Analysis of Recursive Algorithms:**
 - **Recurrence Equations**
 - **Solving Recurrence Equations**
 - **Master's Theorem**
 - **Iteration Method**
 - **Recursion Tree Method**
 - **Substitution method**
- **AVL Tree: Rotations, Insertion, Deletion**
- **Algorithm:** An algorithm is a finite set of instructions that accomplishes a particular task.
- **Characteristics of an Algorithm**
 - **Input:** Zero or more inputs are externally supplied.
 - **Output:** At least one output is produced.
 - **Definiteness:** Each instruction is clear and unambiguous.
 - “add 6 or 7 to x”, “compute 5/0” etc. are not permitted.
 - **Finiteness:** The algorithm terminates after a finite number of steps.
 - **Effectiveness:** Every instruction must be very basic so that it can be carried out by a person using only pencil and paper in a finite amount of time. It also must be feasible.
- **Computational Procedures**
 - Algorithms those are definite and effective.
 - Example: Operating system of a digital computer. (When no jobs are available, it does not terminate but continues in a waiting state until a new job is entered.)
- **Program:** It is the expression of an algorithm in a programming language
- **Recursive Algorithms**
 - A recursive function is a function that is defined in terms of itself.
 - An algorithm is said to be recursive if the same algorithm is invoked in the body.
 - Two types of recursive algorithms
 - **Direct Recursion:** An algorithm that calls itself is direct recursive.
 - **Indirect Recursion:** Algorithm A is said to be indirect recursive if it calls another algorithm which in turn calls A.
- **Performance Analysis(Criteria for Analysing Algorithms)**
 - Performance analysis depends on **Space Complexity** and **Time Complexity**

- **Space Complexity**
 - The space complexity of an algorithm is the amount of memory it needs to run to completion
 - Space Complexity = Fixed Part + Variable Part
 $S(P) = c + S_p$, Where P is any algorithm
 - A fixed part:
 - It is independent of the characteristics of the inputs and outputs.
 - Eg:
 - Instruction space(i.e., space for the code)
 - space for simple variables and fixed-size component variables
 - space for constants
 - A variable part:
 - It is dependent on the characteristics of the inputs and outputs.
 - Eg:
 - Space needed by component variables whose size is dependent on the particular problem instance being solved
 - Space needed by referenced variables
 - Recursion stack space.
- **Time Complexity**
 - The time complexity of an algorithm is the amount of computer time it needs to run to completion. Compilation time is excluded.
 - Time Complexity = Frequency Count * Time for Executing one Statement
 - Frequency Count → Number of times a particular statement will execute

- Eg1: Find the time and space complexity of matrix addition algorithm

	Step/Execution	Frequency Count	Total Frequency Count
Algorithm mAdd(m,n,a,b,c)	0	0	0
{	0	0	0
for i=1 to m do	1	m+1	m+1
for j=1 to n do	1	m(n+1)	mn+m
c[i,j] := a[i,j] + b[i,j];	1	mn	mn
}	0	0	0
			2mn + 2m + 1

Time Complexity = $2mn + 2m + 1$

Space Complexity = Space for parameters and Space for local variables
 $m \rightarrow 1$ $n \rightarrow 1$ $a[] \rightarrow mn$ $b[] \rightarrow mn$ $c[] \rightarrow mn$ $i \rightarrow 1$ $j \rightarrow 1$

Space complexity = $3mn + 4$

- Eg2: Find the time and space complexity of recursive sum algorithm

	Step/Execution	Frequency Count		Total Frequency Count	
		$n \leq 0$	$n > 0$	$n \leq 0$	$n > 0$
Algorithm RSum(a,n)	0	0	0	0	0
{	0	0	0	0	0
if $n \leq 0$ then	1	1	1	1	1
return 0	1	1	0	1	0
Else	0	0	0	0	0
return $a[n] + \text{RSum}(a,n-1)$	$1 + T(n-1)$	0	1	0	$1 + T(n-1)$
}	0	0	0	0	0
				2	$2 + T(n-1)$

$$\text{Time Complexity} = T(n) = \begin{cases} 2 & \text{if } n \leq 0 \\ 2 + T(n-1) & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 2 + T(n-1) \\ &= 2 + 2 + T(n-2) \\ &= 2 + 2 + 2 + T(n-3) \\ &= 2 \times 3 + T(n-3) \\ &\vdots \\ &= 2 \times n + T(n-n) \\ &= 2n + 2 \end{aligned}$$

Space Complexity = Space for Stack

= Space for parameters + Space for local variables + Space for return address

For each recursive call the amount of stack required is 3

Space for parameters: $a \rightarrow 1$ $n \rightarrow 1$

Space for local variables: No local variables

Space for return address: 1

Total number of recursive call = $n+1$

Space complexity = $3(n+1)$

• University Questions:

- Discuss the time complexity of the following three functions

```
int fun1(int n)
{
    if( $n \leq 1$ )
        return n;
    return 2xfun1(n-1);
}

int fun2(int n)
{
    if( $n \leq 1$ )
        return n;
    return fun2(n-1)xfun2(n-1);
}
```

Answer:

	Frequency Count $n \leq 1$	Frequency Count $n > 1$
int fun1(int n)		
{ if(n ≤ 1) —————→	1	1
return n; —————→	1	0
return 2xfun1(n-1); —————→	0	1+T(n-1)
}		

2 2 + T(n-1)

$$\text{Time Complexity} = T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ 2+T(n-1) & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 2 + T(n-1) \\ &= 2 + [2 + T(n-2)] = 2 \times 2 + T(n-2) \\ &= 2 \times 2 + [2 + T(n-3)] = 3 \times 2 + T(n-3) \\ &\dots\dots\dots \\ &= k \times 2 + T(n-k) \end{aligned}$$

Assume that $n-k=1 \rightarrow k=n-1$

$$\begin{aligned} T(n) &= 2(n-1) + T(1) \\ &= 2(n-1) + 2 = 2n \\ &= \mathbf{O(n)} \end{aligned}$$

	Frequency Count $n \leq 1$	Frequency Count $n > 1$
int fun2(int n)		
{ if(n ≤ 1) —————→	1	1
return n; —————→	1	0
return fun2(n-1)xfun2(n-1); —————→	0	1+2T(n-1)
}		

2 2 + 2T(n-1)

$$\text{Time Complexity} = T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ 2+2T(n-1) & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 2 + 2T(n-1) \\ &= 2 + 2[2 + 2T(n-2)] = 2 + 2^2 + 2^2T(n-2) \\ &= 2 + 2^2 + 2^2[2 + 2T(n-3)] = 2 + 2^2 + 2^3 + 2^3T(n-3) \\ &\dots\dots\dots \\ &= 2 + 2^2 + 2^3 + \dots + 2^k + 2^kT(n-k) \end{aligned}$$

Assume that $n-k=1 \rightarrow k=n-1$

$$\begin{aligned} T(n) &= 2[1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1}] + 2^kT(n-k) \\ &= 2[(2^k - 1)/(2 - 1)] + 2^kT(1) \\ &= 2^{k+1} - 2 + 2^k \times 2 = 2^{k+1} - 2 + 2^{k+1} \\ &= 2^n - 2 + 2^n = 2 \times 2^n - 2 = \mathbf{O(2^n)} \end{aligned}$$

2. Analyze the complexity of the following program

```
int fun3(int n)
{
    if(n ≤ 1)
        return n;
    return fun3(n-1) + fun3(n-1);
}
```

Answer:

	Frequency Count n ≤ 1	Frequency Count n > 1
int fun3(int n)		
{ if(n ≤ 1)	1	1
return n;	1	0
return fun3(n-1)+fun3(n-1);	0	1+2T(n-1)
}		

$$\text{Time Complexity} = T(n) = \begin{cases} 2 & \text{if } n \leq 1 \\ 2 + 2T(n-1) & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 2 + 2T(n-1) \\ &= 2 + 2[2 + 2T(n-2)] = 2 + 2^2 + 2^2T(n-2) \\ &= 2 + 2^2 + 2^2[2 + 2T(n-3)] = 2 + 2^2 + 2^3 + 2^3T(n-3) \\ &\dots\dots\dots \\ &= 2 + 2^2 + 2^3 + \dots + 2^k + 2^kT(n-k) \end{aligned}$$

Assume that $n-k=1 \rightarrow k=n-1$

$$\begin{aligned} T(n) &= 2[1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1}] + 2^kT(n-k) \\ &= 2[(2^k - 1)/(2 - 1)] + 2^kT(1) \\ &= 2^{k+1} - 2 + 2^k \times 2 = 2^{k+1} - 2 + 2^{k+1} \\ &= 2^n - 2 + 2^n = 2 \times 2^n - 2 = \mathbf{O(2^n)} \end{aligned}$$

3. Analyze the complexity of the following program

```
main()
{
    for(i=1; i ≤ n; i=i*2)
        sum = sum + i + func(i);
}
void func(int m)
{
    for(j=1; j ≤ m; j++)
        Statement with O(1) complexity
}
```

Answer:

main()	
{ for(i=1; i ≤ n; i=i*2)	
sum = sum + i + func(i);	
}	
void func(int m)	
{ for(j=1; j ≤ m; j++)	
Statement with O(1) complexity	
}	Most executing statements

for loop in main() will successfully execute in $\log_2(n)$ times

for loop in func() will execute $2^0, 2^1, 2^2, \dots, 2^{\log(n)-1}$ times

$$\begin{aligned} T(n) &= 2^0 + 2^1 + 2^2 + \dots + 2^{\log(n)-1} \\ &= (2^{\log(n)} - 1)/(2 - 1) = 2^{\log(n)} - 1 \\ &= n^{\log(2)} - 1 \\ &= n - 1 = \mathbf{O(n)} \end{aligned}$$

4. Analyse the complexity of the following function

```
void function(int n)
{
    int count=0;
    for(int i=n/2; i<=n; i++)
        for(int j=1; j<=n; j=2*j)
            for(int k=1; k<=n; k=k*2)
                count++;
}
```

Answer:

	Frequency Count	
void function(int n)		
{ int count=0;	→ 1	=O(1)
for(int i=n/2; i<=n; i++)	→ (n/2)+1	=O(n)
for(int j=1; j<=n; j=2*j)	→ (n/2)log(n) + 1	=O(nlog(n))
for(int k=1; k<=n; k=k*2)	→ (n/2)log(n)log(n) + 1	=O(nlog(n)log(n))
count++;	→ (n/2)log(n)log(n)	=O(nlog(n)log(n))
}		

O(nlog(n)log(n))

Time Complexity = O(nlog(n)log(n))

5. Analyse the complexity of the following functions

```
function(int n)
{
    if(n==1) return;
    for(i=1; i<=n; i++)
        for(int j=1; j<=n; j++)
            {
                printf("*");
                break;
            }
}

void function(int n)
{
    int i=1; s=1;
    while(s<=n)
    {
        i++;
        s+=i;
        printf("*");
    }
}
```

Answer:

	Frequency Count
function(int n)	
{ if(n==1) return; }	1 =O(1)
for(i=1; i<=n; i++)	n+1 =O(n)
for(int j=1; j<=n; j++)	n =O(n)
{ printf("*");	n =O(n)
break;	n =O(n)
}	
}	
	O(n)

Time Complexity = T(n) = O(n)

void function(int n)	
{ int i=1, s=1; }	O(1)
while(s<=n)	
{ i++;	
s+=i;	
printf("*");	
}	
}	
	Most executing statements

For each iteration i will be incremented by one and added to S
 $1+2+3+\dots+k \leq n$, where k is the maximum number of iterations of while loop

$$[k(k+1)]/2 \leq n \rightarrow (k^2+k)/2 \leq n \rightarrow k \approx \sqrt{n}$$

Time Complexity = $O(\sqrt{n})$

6. Express the return value of the function “mystery” in θ notation

```
int mystery(int n)
{
    int j=0,total=0;
    for (int i=1;j<=n;i++)
    {
        ++total;
        j+=2*i;
    }
    return total;
}
```

Answer:

The value of total is depends on the number of iterations of for loop
 $0+2 \times 1+2 \times 2+2 \times 3+ \dots +2 \times (k-1) \leq n$, where k is the number of iterations of for loop

$$2(1+2+3+\dots+(k-1)) \leq n$$

$$2[(k-1)k]/2 \leq n \rightarrow [k^2-k] \leq n \rightarrow k \approx \sqrt{n}$$

Time Complexity = $\theta(\sqrt{n})$

7. Find the time complexity of the given code segment

```
for(int i=1;i<=n;i++)
{
    for(int j=i+1;j<=n;j++)
    {
        //code
    }
}
```

Answer: The most frequently executing statement is the innermost code. During the first iteration of outer loop that code will execute n-1 times. During the 2nd iteration of outer loop that code will execute n-2 times and so on. During the last iteration it will never execute.

So the frequency count of that code = (n-1) + (n-2) + + 0
= (n-1)n/2 = (n²-n)/2

Time Complexity = **O(n²)**

8. Find the time complexity of the given code segment

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i*i;j++)
    {
        for(int k=1;k<=n/2;k++)
        {
            //code
        }
    }
}
```

Answer: The 1st loop will successfully execute n times

The 2nd loop will successfully execute 1² + 2² + 3² + + n² times

The 3rd loop will successfully execute (1² + 2² + 3² + + n²) n / 2 times

So the frequency count of innermost code = (1² + 2² + 3² + + n²) n / 2
= [n(n+1)(2n+1)/6] n / 2

Time Complexity = **O(n⁴)**

9. Find the time complexity of the given code segment

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=i;j++)
    {
        for(int k=1;k<=100;k++)
        {
            //code
        }
    }
}
```

Answer: The 1st loop will successfully execute n times

The 2nd loop will successfully execute 1 + 2 + 3 + + n times

The 3rd loop will successfully execute (1 + 2 + 3 + + n)100 times

So the frequency count of innermost code = (1 + 2 + 3 + + n)100
= [n(n+1)/2] 100

Time Complexity = **O(n²)**

10. Consider the following C function

```
int check(int n)
{
    int i,j;
    for (i=1;i<=n;i++)
    {
        for (j=1;j<n;j+=i)
        {
            printf("%d",i+j);
        }
    }
}
```

Find the time complexity of check in terms of θ notation

- **Best Case, Worst Case and Average Case Complexity**

- In certain case we cannot find the exact value of frequency count. In this case we have 3 types of frequency counts
 - Best Case : It is the minimum number of steps that can be executed for a given parameter
 - Worst Case: It is the maximum number of steps that can be executed for a given parameter
 - Average Case: It is the average number of steps that can be executed for a given parameter
- Example: Linear Search
 - Best Case: Search data will be in the first location of the array.
 - Worst Case: Search data does not exist in the array
 - Average Case: Search data is in the middle of the array.

	Best Case			Worst Case			Average Case		
	S/E	FC	TFC	S/E	FC	TFC	S/E	FC	TFC
Algorithm Search(a,n,x)	0	0	0	0	0	0	0	0	0
{	0	0	0	0	0	0	0	0	0
for i:=1 to n do	1	1	1	1	n+1	n+1	1	n/2	n/2
if a[i] ==x then	1	1	1	1	n	n	1	n/2	n/2
return i;	1	1	1	1	0	0	1	1	1
return -1;	1	0	0	1	1	1	1	0	0
}	0	0	0	0	0	0	0	0	0
			3				2n + 2		
							n+1		

Best Case Complexity = 3 = $\Omega(1)$

Worst Case Complexity = 2n + 2 = $O(n)$

Average Case Complexity = n+1 = $\Theta(n)$

- **University Questions:**

1. Define the terms best case, average case and worst case complexities.
2. Discuss the best case and worst case time complexity of linear search algorithm
3. Determine the Best case and Worst case time complexity of the following function

```
void fun(int n, int arr[])
{
    int i=0; j=0;
    for(;i<n;++i)
        while(j<n && arr[i]< arr[j])
            j++;
}
```

4. Illustrate best case, average case and worst case complexity of Insertion sort algorithm.

Answer:

```
Algorithm InsertionSort(A,n)
{
    for i=1 to n-1 do
    {
        j=i
        while j>0 and A[j-1] > A[j] do
        {
            Swap(A[j], A[j-1])
            j=j-1
        }
    }
}
```

Best case complexity: Best case situation occurs when the array itself is in sorted order. In this case the while loop will not execute successfully. So the time complexity is proportional to number of times the for loop will execute. It will execute $O(n)$ time.

The best case time complexity = $\Omega(n)$

Worst case complexity: Worst case situation occurs when the array itself is in reverse sorted order. In this case the while loop will execute $1+2+3+ \dots +(n-1)=n(n+1)/2$ times.

The time complexity is proportional to number of times the while loop will execute. It will execute $O(n^2)$ time.

The worst case time complexity = $O(n^2)$

Average case complexity: Average case situation occurs when the while loop will iterate half of its maximum iterations. In this case the while loop will execute $[1+2+3+ \dots +(n-1)]/2=n(n+1)/4$ times.

The time complexity is proportional to number of times the while loop will execute. It will execute $O(n^2)$ time.

The average case time complexity = $\Theta(n^2)$

5. Write an algorithm for insertion sort. Calculate the worst case time complexity of insertion sort.

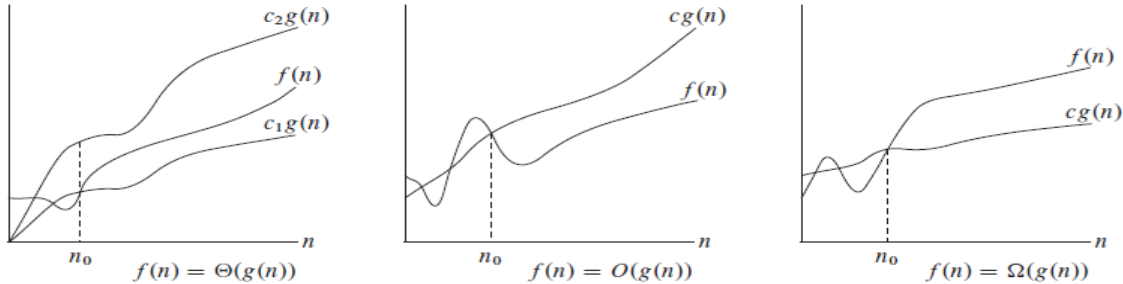
- **Asymptotic Notations**

- It is the mathematical notations to represent frequency count. 5 types of asymptotic notations

- **Big Oh (O)**

- The function $f(n) = O(g(n))$ iff there exists 2 positive constants c and n_0 such that $0 \leq f(n) \leq c g(n)$ for all $n \geq n_0$
- It is the measure of longest amount of time taken by an algorithm(Worst case).
- It is asymptotically tight upper bound
- $O(1)$: Computational time is constant
- $O(n)$: Computational time is linear
- $O(n^2)$: Computational time is quadratic
- $O(n^3)$: Computational time is cubic
- $O(2^n)$: Computational time is exponential

- **Omega (Ω)**
 - The function $f(n) = \Omega(g(n))$ iff there exists 2 positive constant c and n_0 such that $f(n) \geq c g(n) \geq 0$ for all $n \geq n_0$
 - It is the measure of smallest amount of time taken by an algorithm(Best case).
 - It is asymptotically tight lower bound
- **Theta (Θ)**
 - The function $f(n) = \Theta(g(n))$ iff there exists 3 positive constants c_1, c_2 and n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$
 - It is the measure of average amount of time taken by an algorithm(Average case).



- **Little Oh (o)**
 - The function $f(n) = o(g(n))$ iff for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq f(n) < c g(n)$ for all $n \geq n_0$
 - It is asymptotically loose upper bound

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$g(n)$ becomes arbitrarily large relative to $f(n)$ as n approaches infinity

- **Little Omega (ω)**
 - The function $f(n) = \omega(g(n))$ iff for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $f(n) > c g(n) \geq 0$ for all $n \geq n_0$
 - It is asymptotically loose lower bound

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

$f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity

- **Examples:**

1. Find the O notation of the following functions

a) $f(n) = 3n + 2$

$$3n + 2 \leq 4n \quad \text{for all } n \geq 2$$

Here $f(n) = 3n + 2$ $g(n) = n$

$$c = 4$$

$$n_0 = 2$$

Therefore $3n + 2 = \mathbf{O(n)}$

b) $f(n) = 4n^3 + 2n + 3$

$$4n^3 + 2n + 3 \leq 5n^3 \quad \text{for all } n \geq 2$$

Here $f(n) = 4n^3 + 2n + 3$

$$g(n) = n^3$$

$$c = 5$$

$$n_0 = 2$$

Therefore $4n^3 + 2n + 3 = \mathbf{O(n^3)}$

c) $f(n) = 2^{n+1}$

$$2^{n+1} \leq 2 \cdot 2^n \quad \text{for all } n \geq 1$$

- Here $f(n) = 2^{n+1}$ $g(n) = 2^n$ $c=2$ $n_0=1$
 Therefore $2^{n+1} = O(2^n)$
- d) $f(n) = 2^n + 6n^2 + 3n$
 $2^n + 6n^2 + 3n \leq 7 \cdot 2^n$ for all $n \geq 5$
 Here $f(n) = 2^n + 6n^2 + 3n$ $g(n) = 2^n$ $c=7$ $n_0=5$
 Therefore $2^n + 6n^2 + 3n = O(2^n)$
- e) $f(n) = 10n^2 + 7$
 f) $f(n) = 5n^3 + n^2 + 6n + 2$
 g) $f(n) = 6n^2 + 3n + 2$
 h) $f(n) = 100n + 6$

2. Is $2^{2n} = O(2^n)$? (Univ Question)
 $2^{2n} \leq c \cdot 2^n$
 $2^n \leq c$
 There is no value for c and n_0 that can make this true.
 Therefore $2^{2n} \neq O(2^n)$

3. Is $2^{n+1} = O(2^n)$? (Univ Question)
 $2^{n+1} \leq c \cdot 2^n$
 $2 \cdot 2^n \leq c \cdot 2^n$
 $2 \leq c$
 $2^{n+1} \leq c \cdot 2^n$ is True if $c=2$ and $n \geq 1$.
 Therefore $2^{n+1} = O(2^n)$

4. Let $f(n) = 3n^3 + 2n^2 + 3$ for an algorithm. Let $g(n) = n^3$. Prove that $f(n)$ of this algorithm is in $O(n^3)$. (Univ Question)

Answer: $f(n) = 3n^3 + 2n^2 + 3$
 Assume that $g(n) = n^3$ and $c=10$
 As per O notation $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
 $0 \leq 3n^3 + 2n^2 + 3 \leq c \cdot n^3$ for all $n \geq 1$
 Therefore, $f(n) = O(g(n)) = O(n^3)$

5. What is the smallest value of n such that an algorithm whose running time is $100n^2$ runs faster than an algorithm whose running time is 2^n on the same machine? (Univ Question)

6. Find the Ω notation of the following functions

- a) $f(n) = 27n^2 + 16n + 25$
 $27n^2 + 16n + 25 \geq 27n^2$ for all $n \geq 1$
 Here $c=27$ $n_0=1$ $g(n) = n^2$
 $27n^2 + 16n + 25 = \Omega(n^2)$
- b) $f(n) = 5n^3 + n^2 + 3n + 2$
 $5n^3 + n^2 + 3n + 2 \geq 5n^3$ for all $n \geq 1$
 Here $c=5$ $n_0=1$ $g(n) = n^3$
 $5n^3 + n^2 + 3n + 2 = \Omega(n^3)$
- c) $f(n) = 3^n + 6n^2 + 3n$
 $3^n + 6n^2 + 3n \geq 5 \cdot 3^n$ for all $n \geq 1$
 Here $c=5$ $n_0=1$ $g(n) = 3^n$
 $3^n + 6n^2 + 3n = \Omega(3^n)$

- d) $f(n) = 4 \cdot 2^n + 3n$
- e) $f(n) = 3n + 30$
- f) $f(n) = 10n^2 + 4n + 2$

7. Find the Θ notation of the following functions

a) $f(n) = 3n + 2$

$$3n + 2 \leq 4n \quad \text{for all } n \geq 2$$

$$3n + 2 = O(n)$$

$$3n + 2 \geq 3n \quad \text{for all } n \geq 1$$

$$3n + 2 = \Omega(n)$$

$$3n \leq 3n + 2 \leq 4n \quad \text{for all } n \geq 2$$

$$3n + 2 = \Theta(n)$$

b) $f(n) = 3 \cdot 2^n + 4n^2 + 5n + 2$

$$3 \cdot 2^n + 4n^2 + 5n + 2 \leq 10 \cdot 2^n \quad \text{for all } n \geq 1$$

$$3 \cdot 2^n + 4n^2 + 5n + 2 = O(2^n)$$

$$3 \cdot 2^n + 4n^2 + 5n + 2 \geq 3 \cdot 2^n \quad \text{for all } n \geq 1$$

$$3 \cdot 2^n + 4n^2 + 5n + 2 = \Omega(2^n)$$

$$3 \cdot 2^n \leq 3 \cdot 2^n + 4n^2 + 5n + 2 \leq 10 \cdot 2^n \quad \text{for all } n \geq 1$$

$$3 \cdot 2^n + 4n^2 + 5n + 2 = \Theta(2^n)$$

c) $f(n) = 2n^2 + 16$

d) $f(n) = 27n^2 + 16$

• Common Complexity Functions

• Constant Time

- An algorithm is said to be constant time if the value of $f(n)$ is bounded by a value that does not depend on the size of input.
- Computational time is constant
- Eg: $O(1)$

• Logarithmic Time

- An algorithm is said to be logarithmic time if $f(n) = O(\log n)$

• Linear Time

- If $f(n) = O(n)$, then the algorithm is said to be linear time .

• Quadratic Time

- If $f(n) = O(n^2)$, then the algorithm is said to be quadratic time .

• Polynomial Time

- If $f(n) = O(n^k)$, then the algorithm is said to be polynomial time .

• Exponential Time

- If $f(n) = O(2^n)$, then the algorithm is said to be exponential time .

• Factorial Time

- If $f(n) = O(n!)$, then the algorithm is said to be factorial time

- **Running Time Comparison (Classifying functions by their asymptotic growth rate)**

- Logarithmic functions are very slow
- Exponential functions and factorial functions are very fast growing

n	log n	n	n log n	n^2	n^3	2^n	n!
10	3.3	10	3.3×10	10^2	10^3	10^3	3.6×10^{60}
10^2	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
10^3	10	10^3	10×10^3	10^6	10^9	.	.
10^4	13	10^4	13×10^4	10^8	10^{12}	.	.
10^5	17	10^5	17×10^5	10^{10}	10^{15}	.	.
10^6	20	10^6	20×10^6	10^{12}	10^{18}	.	.

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^k) < O(2^n) < O(n!)$$

- **Properties of Asymptotic Notations**

- Reflexivity
 - $f(n) = O(f(n))$
 - $f(n) = \Omega(f(n))$
 - $f(n) = \Theta(f(n))$
- Symmetry
 - $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$
- Transpose Symmetry
 - $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$
 - $f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$
- Transitivity
 - $f(n) = O(g(n))$ and $g(n) = O(h(n))$ imply $f(n) = O(h(n))$
 - $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ imply $f(n) = \Omega(h(n))$
 - $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ imply $f(n) = \Theta(h(n))$
 - $f(n) = o(g(n))$ and $g(n) = o(h(n))$ imply $f(n) = o(h(n))$
 - $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ imply $f(n) = \omega(h(n))$

- **University Questions:**

1. Explain asymptotic notations in algorithm analysis
2. Define Big Oh, Big Omega and Theta notations and illustrate them graphically.
3. What do you meant by asymptotic growth rate? Define Big Oh, Big Omega and Theta notations
4. Define asymptotic notation. Arrange the following functions in increasing order of asymptotic growth rate. $n^3, 2^n, \log n^3, 2^{100}, n^2 \log n, n^n, \log n, n^{0.3}, 2^{\log n}$

Answer : $2^{100} < \log n < \log n^3 < n^{0.3} < n^2 \log n < n^3 < 2^{\log n} < 2^n < n^n$

- **Recurrence Relations**

- A recurrence is an equation or inequality that describes a function in terms of its values on smaller inputs.
- There are several methods for solving recurrence relation
 - **Iteration Method**
 - **Recursion tree Method**
 - **Substitution Method**
 - **Master's Method**

- **Iteration Method**

- Solve the following recurrence relation using Iteration method.

1. $T(n) = 1 + T(n-1)$

$$T(n) = 1 + T(n-1)$$

$$= 1 + 1 + T(n-2) = 2 + T(n-2)$$

$$= 2 + 1 + T(n-3) = 3 + T(n-3)$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$= k + T(n-k) \quad k^{\text{th}} \text{ term}$$

$$\text{Assume } n-k = 1$$

$$\rightarrow k = n-1$$

$$T(n) = n-1 + T(n-(n-1))$$

$$= n-1 + T(1) = O(n) + O(1) = \mathbf{O(n)}$$

2. $T(2^k) = 3 T(2^{k-1}) + 1$

(Univ Question)

$$T(1) = 1$$

Answer:

$$T(2^k) = 1 + 3T(2^{k-1})$$

$$= 1 + 3[1 + 3T(2^{k-2})] = 1 + 3 + 3^2 T(2^{k-2})$$

$$= 1 + 3 + 3^2 [1 + 3T(2^{k-3})] = 1 + 3 + 3^2 + 3^3 T(2^{k-3})$$

.....

$$= 1 + 3 + 3^2 + \dots + 3^{n-1} + 3^n T(2^{k-n})$$

$n^{\text{th}} \text{ term}$

$$\text{Assume that } 2^{k-n}=1$$

$$\rightarrow 2^k/2^n=1$$

$$\rightarrow$$

$$2^k=2^n \rightarrow k=n$$

$$T(2^k) = 1 + 3 + 3^2 + \dots + 3^{n-1} + 3^n T(2^{k-n}) = [(3^n - 1)/(3 - 1)] + 3^n T(1)$$

$$= (3^n/2) - (1/2) + 3^n$$

$$= (3/2) 3^n - (1/2)$$

$$= (3/2) 3^k - (1/2)$$

$$= \mathbf{O(3^k)}$$

3. $T(n) = 2T(n/2) + 2$ if $n > 2$
 $= 1$ if $n = 2$

(Univ Question)

Answer:

$$T(n) = 2 + 2T(n/2)$$

$$= 2 + 2[2 + 2T(n/2^2)]$$

$$= 2 + 2^2 + 2^2 T(n/2^2)$$

$$= 2 + 2^2 + 2^2 [2 + 2T(n/2^3)]$$

$$= 2 + 2^2 + 2^3 + 2^3 T(n/2^3)$$

.....

$$= 2 + 2^2 + 2^3 + \dots + 2^k + 2^k T(n/2^k) \quad k^{\text{th}} \text{ term}$$

Assume that $n/2^k=2$

$$\begin{aligned} T(n) &= 2[1+2+2^2+2^3+\dots+2^{k-1}] + 2^k T(n/2^k) \\ &= 2[2^k-1] + 2^k T(2) = 2 \times 2^k - 2 + 2^k \\ &= 3 \times 2^k - 2 = 3 \times (n/2) - 2 \end{aligned}$$

4. $T(n) = 2 T(n/2) + n$ $T(1)=1$ (Univ Question)

Answer:

$$\begin{aligned} T(n) &= n + 2 T(n/2) \\ &= n + 2 [(n/2) + 2 T(n/2^2)] = 2n + 2^2 T(n/2^2) \\ &= 2n + 2^2 [(n/2^2) + 2 T(n/2^3)] = 3n + 2^3 T(n/2^3) \\ &\dots\dots\dots \\ &= k n + 2^k T(n/2^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume that } n/2^k=1 \rightarrow 2^k=n \rightarrow k=\log_2(n) \\ T(n) &= n \log_2(n) + n T(1) \\ &= n \log_2(n) + n \\ &= \mathbf{O(n \log_2(n))} \end{aligned}$$

5. $T(n) = 2$ if $n=1$ (Univ Question)
 $T(n) = 2T(n/2) + 2n + 3$ Otherwise

Answer:

$$\begin{aligned} T(n) &= 3 + 2n + 2T(n/2) \\ &= 3 + 2n + 2[3+2(n/2)+2T(n/2^2)] \\ &= 3+2 \times 3+2 \times 2n+2^2 T(n/2^2) \\ &= 3+2 \times 3+2 \times 2n+2^2 [3+2(n/2^2)+2T(n/2^3)] \\ &= 3+2 \times 3+2^2 \times 3+3 \times 2n+2^3 T(n/2^3) \\ &\dots\dots\dots \\ &= [3+2 \times 3+2^2 \times 3+\dots+2^{k-1} \times 3] + [k \times 2n] + 2^k T(n/2^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume that } n/2^k=1 \rightarrow 2^k=n \rightarrow k=\log_2(n) \\ T(n) &= 3[2^0+2^1+2^2+\dots+2^{k-1}] + 2n \log_2(n) + n T(1) \\ &= 3[(2^k-1)/(2-1)] + 2n \log_2(n) + 2n \\ &= 3(n-1) + 2n \log_2(n) + 2n \\ &= 5n - 3 + 2n \log_2(n) = \mathbf{O(n \log_2(n))} \end{aligned}$$

6. $T(n) = 3 T(n/3) + n$, $T(1) = 1$ (Univ Question)

Answer:

$$\begin{aligned} T(n) &= n + 3T(n/3) \\ &= n + 3[(n/3)+3T(n/3^2)] = n + n + 3^2 T(n/3^2) \\ &= 2n + 3^2 [(n/3^2) + 3T(n/3^3)] \\ &= 3n + 3^3 T(n/3^3) \\ &\dots\dots\dots \\ &= k n + 3^k T(n/3^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume } n/3^k=1 \rightarrow 3^k=n \rightarrow k=\log_3(n) \\ T(n) &= n \log_3(n) + n T(1) \\ &= n \log_3(n) + n = \mathbf{O(n \log_3(n))} \end{aligned}$$

$$7. T(n) = 2 T(n/2) + n^2$$

(Univ Question)

Answer:

$$\begin{aligned}
 T(n) &= n^2 + 2T(n/2) \\
 &= n^2 + 2[(n/2)^2 + 2T(n/2^2)] = n^2 + n^2/2 + 2^2 T(n/2^2) \\
 &= n^2 + n^2/2 + 2^2 [(n/2^2)^2 + 2T(n/2^3)] \\
 &= n^2 + n^2/2 + n^2/2^2 + 2^3 T(n/2^3) \\
 &\dots\dots\dots \\
 &= n^2 [1 + (1/2) + (1/2)^2 + \dots + (1/2)^{k-1}] + 2^k T(n/2^k) \quad k^{\text{th}} \text{ term} \\
 &\quad \text{Assume } n/2^k = 1 \quad \Rightarrow 2^k = n \Rightarrow k = \log_2(n) \\
 T(n) &= n^2 [(1 - (1/2)^k) / (1 - (1/2))] + n T(1) \\
 &= 2 n^2 [1 - (1/2)^k] + n = 2 n^2 [1 - (1/2^{\log_2 n})] + n \\
 &= 2 n^2 [1 - (1/n^{\log_2 2})] + n = 2 n^2 [1 - (1/n)] + n \\
 &= 2 n^2 - 2n + n \\
 &= O(n^2)
 \end{aligned}$$

$$8. T(n) = T(n/3) + n$$

Answer:

$$\begin{aligned}
 T(n) &= n + T(n/3) \\
 &= n + [(n/3) + T(n/3^2)] = n + (n/3) + T(n/3^2) \\
 &= n + (n/3) + [(n/3^2) + T(n/3^3)] = n + (n/3) + (n/3^2) + T(n/3^3) \\
 &\dots\dots\dots \\
 &= n + (n/3) + (n/3^2) + \dots + (n/3^{k-1}) + T(n/3^k) \quad k^{\text{th}} \text{ term} \\
 &\quad \text{Assume } n/3^k = 1 \quad \Rightarrow 3^k = n \Rightarrow k = \log_3(n) \\
 T(n) &= n [1 + (1/3) + (1/3)^2 + \dots + (1/3)^{k-1}] + T(n/3^k) \\
 &= n \left[\frac{1 - (1/3)^k}{1 - (1/3)} \right] + T(1) = \frac{3}{2} n [1 - (1/3^k)] + T(1) \\
 &= \frac{3}{2} n \left[1 - \frac{1}{n} \right] + 1 = \frac{3}{2} n - \frac{3}{2} + 1 \\
 &= O(n)
 \end{aligned}$$

$$9. T(n) = 3 T(n/4) + n$$

Answer:

$$\begin{aligned}
 T(n) &= n + 3T(n/4) \\
 &= n + 3[(n/4) + 3T(n/4^2)] = n + (3/4)n + 3^2 T(n/4^2) \\
 &= n + (3/4)n + 3^2 [(n/4)^2 + 3T(n/4^3)] \\
 &= n + (3/4)n + (3/4)^2 n + 3^3 T(n/4^3) \\
 &\dots\dots\dots \\
 &= n + (3/4)n + (3/4)^2 n + \dots + (3/4)^{k-1} n + 3^k T(n/4^k) \quad k^{\text{th}} \text{ term} \\
 &\quad \text{Assume } n/4^k = 1 \quad \Rightarrow 4^k = n \Rightarrow k = \log_4(n) \\
 T(n) &= n [1 + (3/4) + (3/4)^2 + \dots + (3/4)^{k-1}] + 3^k T(1) \\
 &= n \left[\frac{1 - (3/4)^k}{1 - (3/4)} \right] + 3^k \times 1 \\
 &= 4 n [1 - (3^k/4^k)] + 3^k \quad \mathbf{a^{\log_c(b)} = b^{\log_c(a)}} \\
 &= 4 n [1 - (3^{\log_4 n} / n)] + 3^{\log_4 n} \\
 &= 4 n [1 - (n^{\log_4 3} / n)] + n^{\log_4 3}
 \end{aligned}$$

$$= 4n - 4n^{\log_4 3} + n^{\log_4 3} = 4n - 3n^{\log_4 3} \\ = O(n)$$

10. $T(n) = T(n/2) + 1$

Answer:

$$\begin{aligned} T(n) &= 1 + T(n/2) \\ &= 1 + [1 + T(n/2^2)] = 2 + T(n/2^2) \\ &= 2 + [1 + T(n/2^3)] = 3 + T(n/2^3) \\ &\dots\dots\dots \\ &= k + T(n/2^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume } n/2^k = 1 \quad \Rightarrow 2^k = n \Rightarrow k = \log_2(n) \\ T(n) &= \log_2(n) + T(1) = \log_2(n) + 1 \\ &= O(\log_2(n)) \end{aligned}$$

11. $T(n) = 4T(n/2) + n^2$

Answer:

$$\begin{aligned} T(n) &= n^2 + 4T(n/2) \\ &= n^2 + 4[(n/2)^2 + 4T(n/2^2)] = 2n^2 + 4^2T(n/2^2) \\ &= 2n^2 + 4^2[(n/2^2)^2 + 4T(n/2^3)] = 3n^2 + 4^3T(n/2^3) \\ &\dots\dots\dots \\ &= kn^2 + 4^kT(n/2^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume } n/2^k = 1 \quad \Rightarrow 2^k = n \Rightarrow k = \log_2(n) \\ T(n) &= n^2 \log_2(n) + 4^k T(1) \quad 4^k = n^2 \\ &= n^2 \log_2(n) + n^2 \\ &= O(n^2 \log_2(n)) \end{aligned}$$

12. $T(n) = 4T(n/3) + n$

Answer:

$$\begin{aligned} T(n) &= n + 4T(n/3) \\ &= n + 4[(n/3) + 4T(n/3^2)] = n + (4/3)n + 4^2T(n/3^2) \\ &= n + (4/3)n + 4^2[(n/3^2) + 4T(n/3^3)] \\ &= n + (4/3)n + (4/3)^2n + 4^3T(n/3^3) \\ &\dots\dots\dots \\ &= n + (4/3)n + (4/3)^2n + \dots + (4/3)^{k-1}n + 4^kT(n/3^k) \quad k^{\text{th}} \text{ term} \\ &\quad \text{Assume } n/3^k = 1 \quad \Rightarrow 3^k = n \Rightarrow k = \log_3(n) \\ T(n) &= n[1 + (4/3) + (4/3)^2 + \dots + (4/3)^{k-1}] + 4^kT(1) \\ &= n[((4/3)^k - 1)/((4/3) - 1)] + 4^k \\ &= n[((4^k/3^k) - 1)/(1/3)] + 4^k \\ &= 3n[(4^{\log_3 n}/3^{\log_3 n}) - 1] + 4^{\log_3 n} \\ &= 3n[(n^{\log_3 4}/n^{\log_3 3}) - 1] + n^{\log_3 4} \\ &= 3n[(n^{\log_3 4}/n) - 1] + n^{\log_3 4} \\ &= 3n^{\log_3 4} - 3n + n^{\log_3 4} \\ &= 4n^{\log_3 4} - 3n \\ &= O(n^{\log_3 4}) \end{aligned}$$

- **Recursion Tree Method**

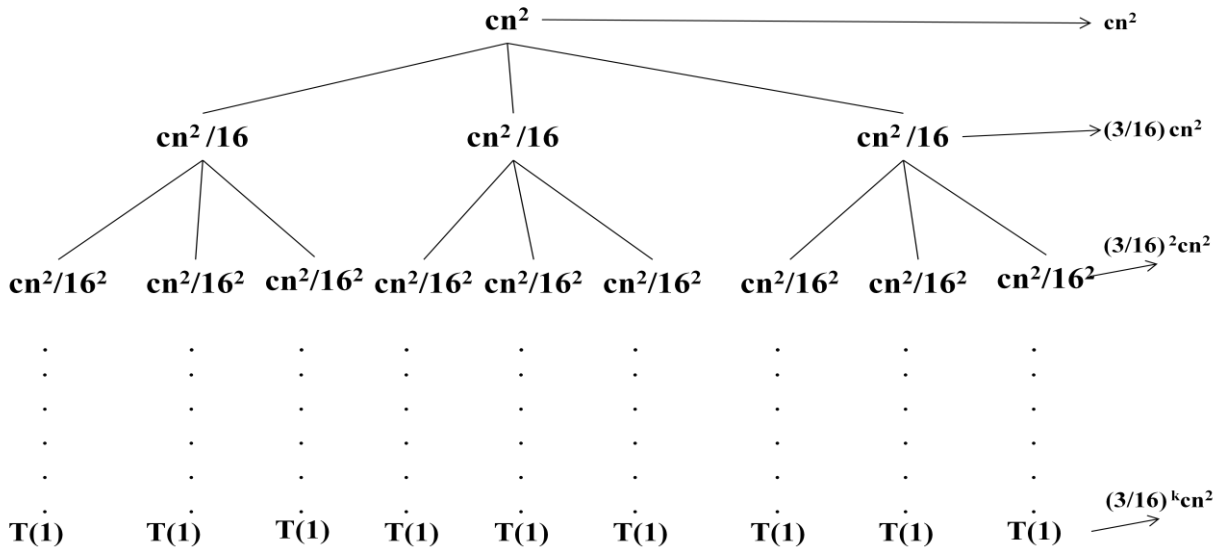
- It is the pictorial representation of iteration method, which is in the form of a tree.
- Solve the following recurrence relation using Recursion Tree method.

1. $T(1) = 1$

(Univ Question)

$$T(n) = 3T(n/4) + cn^2$$

Answer:

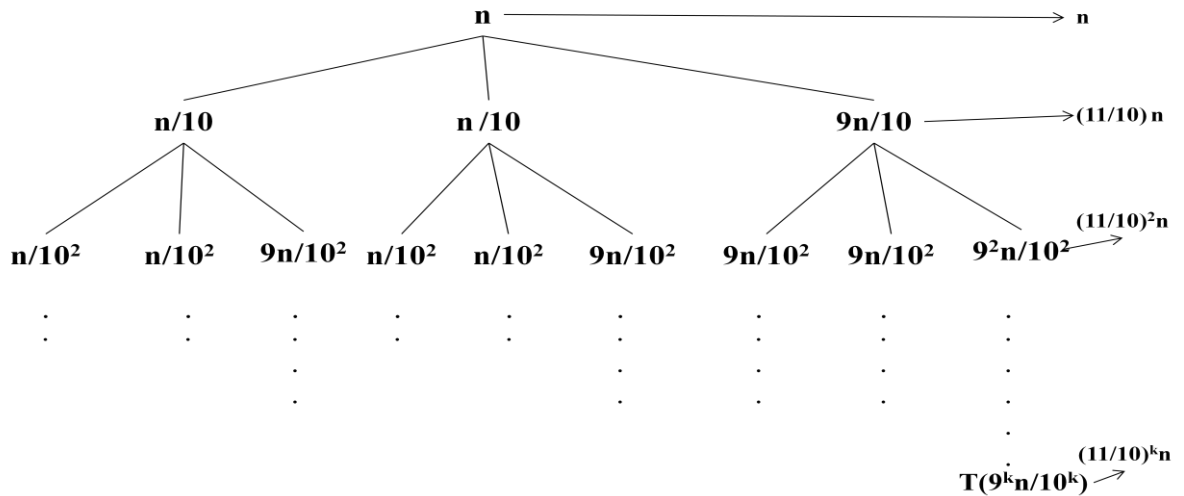


$$\begin{aligned}
T(n) &= n^2 + (3/16)n^2 + (3/16)^2 n^2 + \dots + (3/16)^k n^2 \\
&= n^2 [1 + (3/16) + (3/16)^2 + \dots + (3/16)^k] \\
&\leq n^2 [1 + (3/16) + (3/16)^2 + \dots] \\
&= n^2 [1/(1 - (3/16))] \\
&= (16/13) n^2 \\
&= O(n^2)
\end{aligned}$$

3. $T(n) = 2T(n/10) + T(9n/10) + n$ (Univ Question)

Assume constant time for small value of n

Answer:



Assume that $9^k n / 10^k = 1 \Rightarrow (10/9)^k = n \Rightarrow k = \log_{(10/9)} n$

$$\begin{aligned}
T(n) &= n + (11/10)n + (11/10)^2 n + \dots + (11/10)^k n \\
&= n [1 + (11/10) + (11/10)^2 + \dots + (11/10)^k] \\
&= n [[(11/10)^{k+1} - 1] / [(11/10) - 1]] \\
&= 10n [[(11/10) \times (11/10)^k - 1]] \\
&= 10n [[(11/10) \times (11/10)^{\log_{10/9} n} - 1]] \\
&= 10n [[(11/10) \times n^{\log_{10/9} (11/10)} - 1]] \\
&= 10n [[(11/10) \times n^1] - 1] \\
&= 11n^2 - 10n = O(n^2)
\end{aligned}$$

Longest path: Rightmost path

Longest Path length:

$9^k n / 10^k = 1$, where k is the length of the longest path

$(10/9)^k = n$

$k = \log_{(10/9)} n$

Shortest path: Leftmost path

Shortest Path length:

$n / 10^k = 1$, where k is the length of the shortest path

$10^k = n$

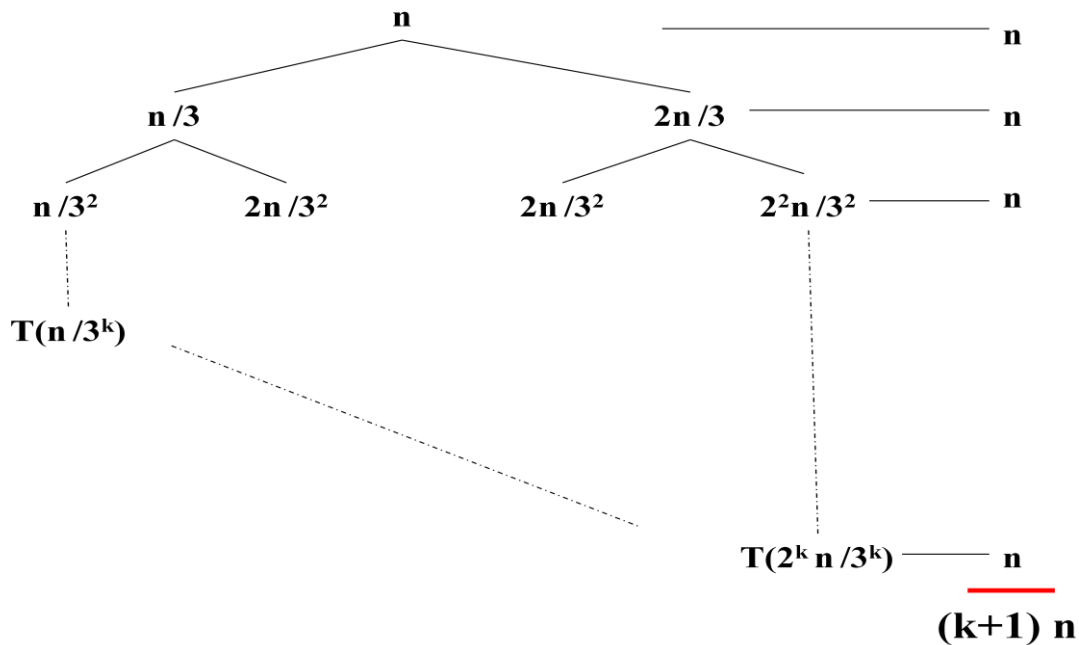
$k = \log_{10} n$

4. $T(n) = T(n/3) + T(2n/3) + n$, where $n > 1$

(Univ Question)

$T(n) = 1$, Otherwise

Answer:



Assume that $2^k n/3^k = 1 \rightarrow (3/2)^k = n \rightarrow k = \log_{(3/2)} n$

$T(n) = (k+1) n$

$= (\log_{(3/2)} n + 1) n$

$= n \log_{(3/2)} n + n$

$= O(n \log_{(3/2)} n)$

Longest path: Rightmost path

Longest Path length:

$2^k n/3^k = 1$, where k is the length of the longest path

$(3/2)^k = n$

$k = \log_{(3/2)} n$

Shortest path: Leftmost path

Shortest Path length:

$n/3^k = 1$, where k is the length of the shortest path

$3^k = n$

$k = \log_3 n$

5. $T(n) = T(n/3) + T(2n/3) + c n$

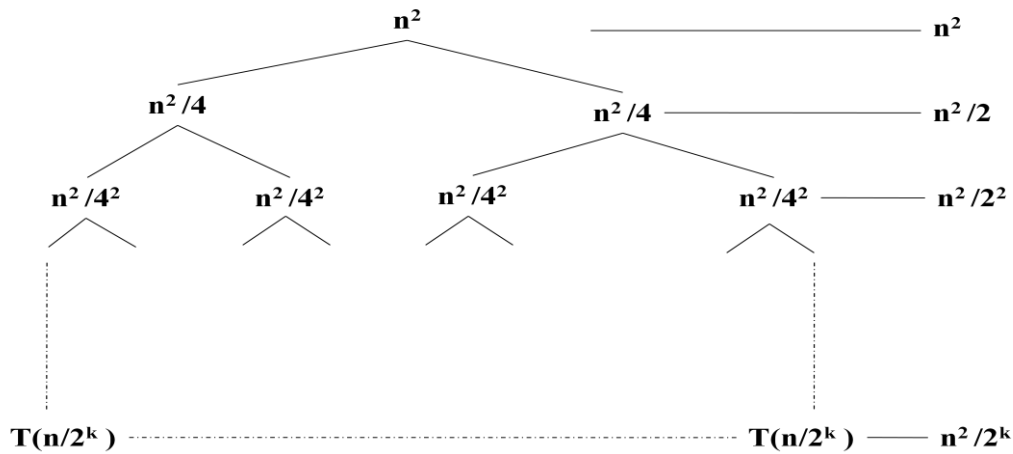
(Univ Question)

6. $T(n) = 2 T(n/2) + n$ where $n > 1$
 0 Otherwise

(Univ Question)

7. $T(n) = 2T(n/2) + n^2$ where $n > 1$ (Univ Question)
 0 otherwise

Answer:

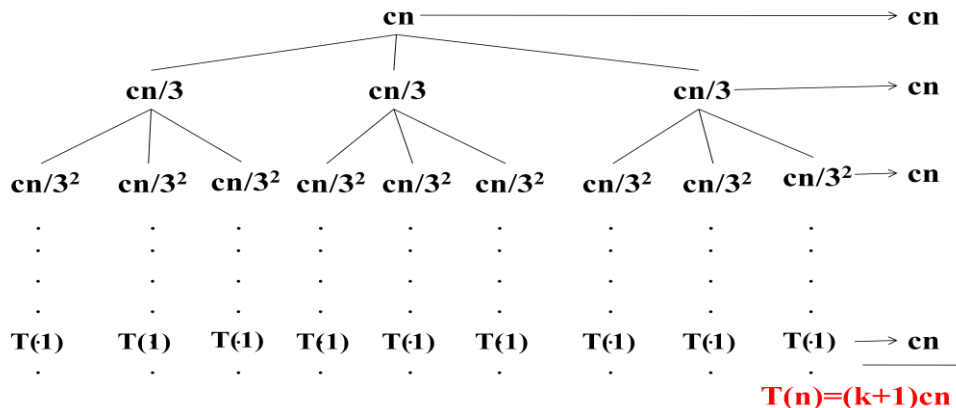


Assume $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

$$\begin{aligned}
 T(n) &= n^2 + (n^2/2) + (n^2/2^2) + \dots + (n^2/2^k) \\
 &= n^2 [1 + (1/2) + (1/2)^2 + \dots + (1/2)^k] \\
 &= n^2 [[1 - (1/2)^{k+1}] / [1 - (1/2)]] \\
 &= 2n^2 [1 - (1/2)^{k+1}] \\
 &= 2n^2 [1 - (1/2)^{k+1}] \\
 &= 2n^2 [1 - (1/2)^{k+1}] \\
 &= 2n^2 - n \\
 &= O(n^2)
 \end{aligned}$$

8. $T(n) = 3T(n/3) + cn$

Answer:

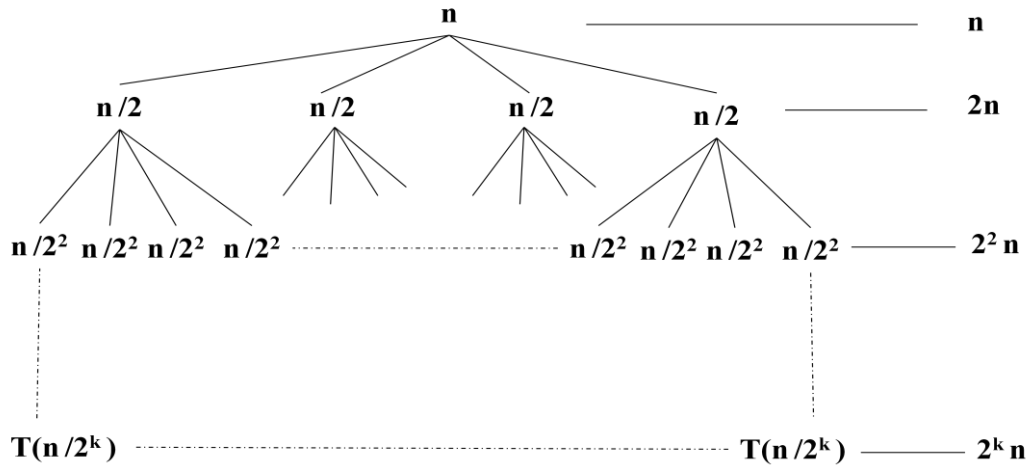


Assume that $n/3^k = 1 \Rightarrow 3^k = n \Rightarrow k = \log_3(n)$

$$\begin{aligned}
 T(n) &= (k+1)cn \\
 &= (\log_3(n) + 1)cn \\
 &= cn \log_3(n) + cn \\
 &= O(n \log_3(n))
 \end{aligned}$$

9. $T(n) = 4 T(n/2) + n$

Answer:

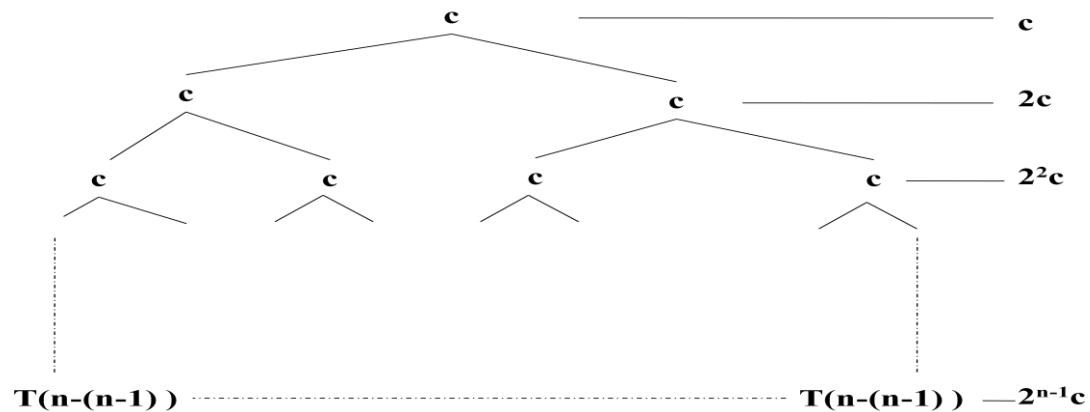


Assume $n/2^k = 1 \rightarrow 2^k = n \rightarrow k = \log_2 n$

$$\begin{aligned}
 T(n) &= n + 2n + 2^2n + \dots + 2^k n \\
 &= n[1 + 2 + 2^2 + \dots + 2^k] \\
 &= n[2^{k+1} - 1] / [2 - 1] \\
 &= n[2 \times 2^k - 1] \\
 &= n[2n - 1] \\
 &= 2n^2 - n \\
 &= O(n^2)
 \end{aligned}$$

10. $T(n) = 2 T(n-1) + c$

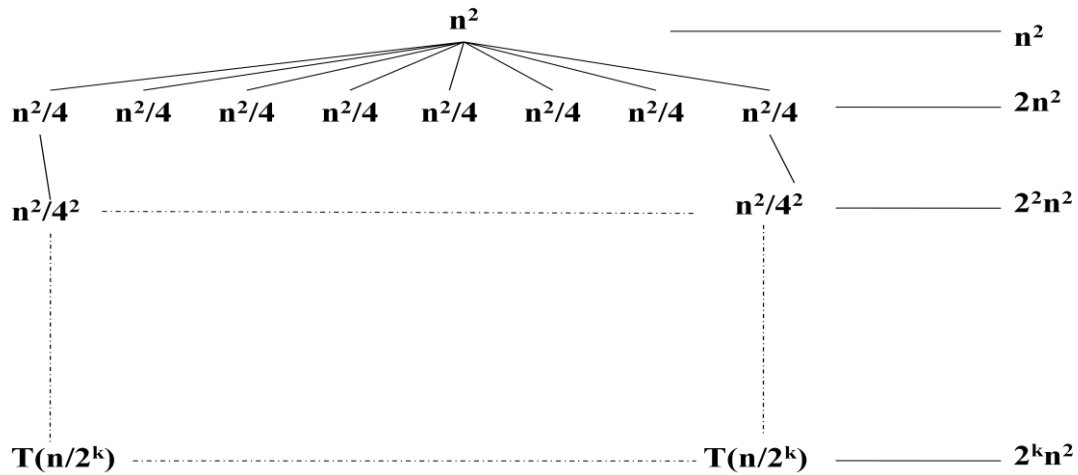
Answer:



$$\begin{aligned}
 T(n) &= c + 2c + 2^2c + \dots + 2^{n-1}c \\
 &= c[1 + 2 + 2^2 + \dots + 2^{n-1}] \\
 &= c[2^n - 1] / [2 - 1] \\
 &= c[2^n - 1] \\
 &= c \cdot 2^n - c \\
 &= O(2^n)
 \end{aligned}$$

11. $T(n) = 8 T(n/2) + n^2$

Answer:

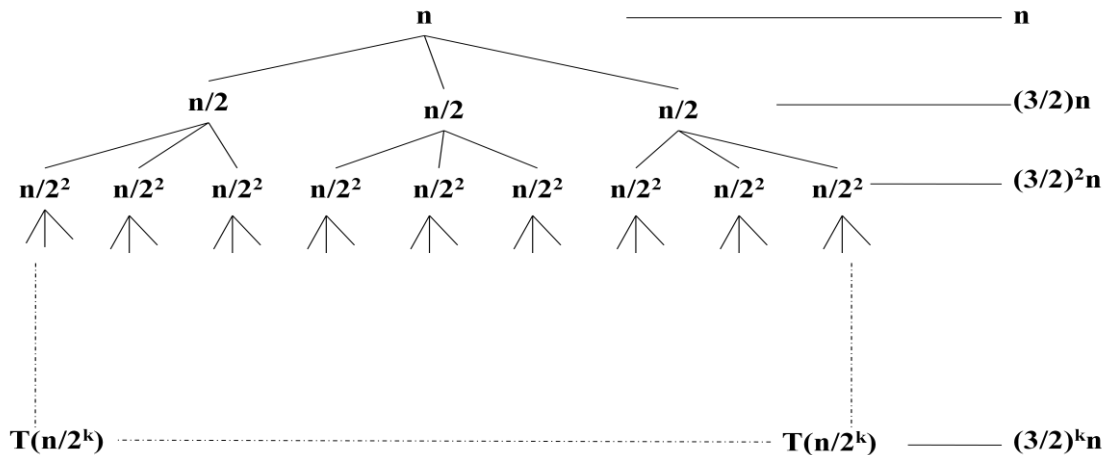


Assume that $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

$$\begin{aligned}
 T(n) &= n^2 + 2n^2 + 2^2n^2 + \dots + 2^kn^2 \\
 &= n^2 [1 + 2 + 2^2 + \dots + 2^k] \\
 &= n^2 [2^{k+1} - 1] / [2 - 1] \\
 &= n^2 [2 \times 2^k - 1] \\
 &= n^2 [2n - 1] \\
 &= 2n^3 - n^2 \\
 &= O(n^3)
 \end{aligned}$$

12. $T(n) = 3T(n/2) + n$

Answer:



Assume that $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

$$\begin{aligned}
 T(n) &= n + (3/2)n + (3/2)^2n + \dots + (3/2)^kn \\
 &= n [1 + (3/2) + (3/2)^2 + \dots + (3/2)^k] \\
 &= n [(3/2)^{k+1} - 1] / [(3/2) - 1]
 \end{aligned}$$

$$\begin{aligned}
&= n \left[\left(\frac{3}{2} \right) \left(\frac{3}{2} \right)^k - 1 \right] / \left[\frac{1}{2} \right] \\
&= 2n \left[\left(\frac{3}{2} \right) \left(\frac{3^k}{2^k} \right) - 1 \right] \\
&= 2n \left[\left(\frac{3}{2} \right) \left(3^{\log_2 n} / 2^{\log_2 n} \right) - 1 \right] \\
&= 2n \left[\left(\frac{3}{2} \right) \left(n^{\log_2 3} / n^{\log_2 2} \right) - 1 \right] \\
&= 2n \left[\left(\frac{3}{2} \right) \left(n^{\log_2 3} / n \right) - 1 \right] \\
&= 3n^{\log_2 3} - 2n \\
&= O(n^{\log_2 3})
\end{aligned}$$

13. $T(n) = T(n/2) + n^2$

Answer:

n^2	_____	n^2
$(n/2)^2$	_____	$n^2/4$
$(n/2^2)^2$	_____	$n^2/4^2$
⋮		
$T(n/2^k)$	_____	$n^2/4^k$

Assume that $n/2^k = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$

$$\begin{aligned}
T(n) &= n^2 + n^2/4 + n^2/4^2 + \dots + n^2/4^k \\
&= n^2 [1 + (1/4) + (1/4)^2 + \dots + (1/4)^k] \\
&= n^2 [[1 - (1/4)^{k+1}] / [1 - (1/4)]] \\
&= n^2 [[1 - (1/4)(1/4)^k] / [3/4]] \\
&= (4/3) n^2 [1 - (1/4)(1/4^{\log_2 n})] \\
&= (4/3) n^2 [1 - (1/4)(1/n^{\log_2 4})] \\
&= (4/3) n^2 [1 - (1/4)(1/n^2)] \\
&= (4/3) n^2 - (1/3) \\
&= O(n^2)
\end{aligned}$$

- **Master's Method**

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k \log^p n)$$

$a \geq 1$ $b > 1$ $k \geq 0$ and p is a real number

1. If $a > b^k$ then $T(n) = \theta(n^{(\log_b a)})$
2. If $a = b^k$
 - a. If $p > -1$ then $T(n) = \theta(n^{(\log_b a)} \log^{p+1}(n))$
 - b. If $p = -1$ then $T(n) = \theta(n^{(\log_b a)} \log(\log n))$
 - c. If $p < -1$ then $T(n) = \theta(n^{(\log_b a)})$
3. If $a < b^k$
 - a. If $p \geq 0$, then $T(n) = \theta(n^k \log^p(n))$
 - b. If $p < 0$, then $T(n) = \theta(n^k)$

- State Master's theorem

(Univ Question)

- Solve the following recurrence relation using Master's method.

1. $T(n) = 3 T(n/2) + n^2$

(Univ Question)

Answer:

$$T(n) = 3 T(n/2) + n^2$$

$$a=3 \quad b=2 \quad n^2 = \Theta(n^2 \log^0(n)) \quad k=2 \quad p=0$$

$$b^k = 2^2 = 4$$

Here $a < b^k$ and $p \geq 0$

$$\begin{aligned} T(n) &= \Theta(n^k \log^p(n)) \\ &= \Theta(n^2 \log^0(n)) \\ &= \Theta(n^2) \end{aligned}$$

2. $T(n) = 2 T(n/2) + n \log n$

(Univ Question)

Answer:

$$T(n) = 2 T(n/2) + n \log n$$

$$a=2 \quad b=2 \quad n \log n = \Theta(n^1 \log^1(n)) \quad k=1 \quad p=1$$

$$b^k = 2^1 = 2$$

Here $a = b^k$ and $p > -1$

$$\begin{aligned} T(n) &= \Theta(n^{(\log_b a)} \log^{p+1}(n)) \\ &= \Theta(n^{(\log_2 2)} \log^2(n)) \\ &= \Theta(n \log^2(n)) \end{aligned}$$

3. $T(n) = 2 T(n/4) + \sqrt{n}$

(Univ Question)

Answer:

$$T(n) = 2 T(n/4) + \sqrt{n}$$

$$a=2 \quad b=4 \quad \sqrt{n} = n^{1/2} = \Theta(n^{1/2} \log^0(n)) \quad k=1/2 \quad p=0$$

$$b^k = 4^{1/2} = 2$$

Here $a = b^k$ and $p > -1$

$$\begin{aligned} T(n) &= \Theta(n^{(\log_b a)} \log^{p+1}(n)) \\ &= \Theta(n^{(\log_4 2)} \log^1(n)) \\ &= \Theta(n^{(1/2)} \log(n)) \\ &= \Theta(\sqrt{n} \log(n)) \end{aligned}$$

4. $T(n) = 7 T(n/2) + n^2$

(Univ Question)

Answer:

$$T(n) = 7T(n/2) + n^2$$

$$a=7 \quad b=2 \quad n^2 = \Theta(n^2 \log^0(n)) \quad k=2 \quad p=0$$

$$b^k = 2^2 = 4$$

Here $a > b^k$

$$T(n) = \Theta(n^{(\log_b a)}) \\ = \Theta(n^{(\log_2 7)})$$

5. $T(n) = 2 T(n/2) + n/\log n$

Answer:

$$T(n) = 2 T(n/2) + (n/\log n)$$

$$a=2 \quad b=2 \quad (n/\log n) = \Theta(n^1 \log^{-1}(n)) \quad k=1 \quad p=-1$$

$$b^k = 2^1 = 2$$

Here $a = b^k$ and $p = -1$

$$T(n) = \Theta(n^{(\log_b a)} \log(\log n)) \\ = \Theta(n^{(\log_2 2)} \log(\log n)) \\ = \Theta(n \log(\log n))$$

6. $T(n) = 3 T(n/4) + n \log n$

(Univ Question)

7. $T(n) = 9 T(n/3) + n$

(Univ Question)

8. $T(n) = 8 T(n/2) + 100 n^2$

(Univ Question)

9. $T(n) = 2 T(n/2) + 10 n$

(Univ Question)

10. $T(n) = 2^n T(n/2) + n^n$

(Univ Question)

Answer:

$$T(n) = 2^n T(n/2) + n^n$$

As per master's theorem, $a \geq 1$. Here $a = 2^n$

So we cannot solve this relation using Master's Method.

11. $T(n) = 4 T(n/2) + n^2$

12. $T(n) = T(n/2) + n^2$

13. $T(n) = 16 T(n/4) + n$

14. $T(n) = 2 T(n/4) + n^{0.51}$

15. $T(n) = 0.5 T(n/2) + 1/n$

16. $T(n) = 6 T(n/3) + n^2 \log n$

17. $T(n) = 64 T(n/8) - n^2 \log n$

18. $T(n) = 7 T(n/3) - n^2$

19. $T(n) = 4 T(n/2) + \log n$

20. $T(n) = \sqrt{2} T(n/2) + \log n$

21. $T(n) = 2 T(n/2) + \sqrt{n}$

22. $T(n) = 3 T(n/2) + n$

23. $T(n) = 3 T(n/3) + \sqrt{n}$

24. $T(n) = 4 T(n/2) + c n$

25. $T(n) = T(2n/3) + 1$

26. $T(n) = 2 T(n/2) + n \log n$

27. $T(n) = 8 T(n/2) + \Theta(n^2)$
 28. $T(n) = 2 T(n/4) + 1$
 29. $T(n) = 2 T(n/4) + n$
 30. $T(n) = 2 T(n/4) + n^2$
 31. $T(n) = T(n/2) + \Theta(1)$
 32. $T(n) = 4 T(n/2) + n^2 \log n$

- **Substitution Method**

- The substitution method for solving recurrences comprises two steps
 1. Guess the form of the solution
 2. Use mathematical induction to find the constant & show that the solution works.
- This method is powerful, but we must be able to guess the form of the answer in order to apply it.

- **Example:**

1. Give the general idea of the substitution method for solving recurrences. Solve the following recurrences using substitution method

$$T(n) = 2 T(n/2) + n \quad T(1) = 1 \quad (\text{Univ Question})$$

Answer:

Guess that $T(n) = O(n \log n)$

As per O notation definition $T(n) \leq c n \log n$

By mathematical induction

$$\begin{aligned} \text{If } n=1, \quad T(1) &\leq c \times 1 \times \log 1 \rightarrow 1 \leq 0 \quad \text{It is false} \\ \text{If } n=2, \quad T(2) &\leq c \times 2 \times \log 2 \rightarrow 2 T(1) + 2 \leq 2c \rightarrow 4 \leq 2c \quad \text{It is true} \\ \text{If } n=3, \quad T(3) &\leq c \times 3 \times \log 3 \rightarrow 2 T(1) + 3 \leq c \times 3 \times \log 3 \\ &5 \leq 3c \log 3 \quad \text{It is true} \end{aligned}$$

This relation is true when $n = 2, 3, 4, \dots, k$

$T(k) \leq c k \log k$, where $2 \leq k \leq n$

$$T(n/2) \leq c (n/2) \log (n/2) \quad \text{—————} \quad \textcircled{1}$$

The recurrence relation is

$$T(n) = 2 T(n/2) + n$$

Apply equation number 1

$$\begin{aligned} T(n) &\leq 2c (n/2) \log (n/2) + n \\ &\leq c n \log (n/2) + n \\ &\leq c n \log n - c n \log 2 + n \\ &\leq c n \log n \end{aligned}$$

Therefore **$T(n) = O(n \log n)$**

2. Solve the following recurrence relation using substitution method

$$\begin{aligned} T(n) &= 1 && \text{if } n=1 \\ &2T(n/2) + 1 && \text{Otherwise} \end{aligned}$$

Answer:

Guess that $T(n) = O(n)$

As per O notation definition $T(n) \leq c n$

By mathematical induction

$$\text{If } n=1, \quad T(1) \leq c \times 1 \rightarrow 1 \leq c \quad \text{It is true}$$

If $n=2$, $T(2) \leq c \times 2 \rightarrow 2 T(1) + 1 \leq 2 c \rightarrow 3 \leq 2c$ It is true

This relation is true when $n= 1, 2, 3, \dots, k$

$T(k) \leq c k$, where $2 \leq k \leq n$

$T(n/2) \leq c (n/2)$ ————— (1)

The recurrence relation is

$$T(n) = 2 T(n/2) + 1$$

Apply equation number 1

$$T(n) \leq 2 c (n/2) + 1$$

$$\leq c n + 1$$

$$\leq c n$$

Therefore **$T(n) = O(n)$**

3. Solve the following recurrences using substitution method

$$T(n) = 2 T(n/2) + k n \quad T(1) = 1$$

Answer:

Guess that $T(n) = O(n \log n)$

As per O notation definition $T(n) \leq c n \log n$

By mathematical induction

$$\text{If } n=1, \quad T(1) \leq c \times 1 \times \log 1 \rightarrow 1 \leq 0 \quad \text{It is false}$$

$$\begin{aligned} \text{If } n=2, \quad T(2) &\leq c \times 2 \times \log 2 \rightarrow 2 T(1) + k \times 2 \leq 2 c \\ &\rightarrow 2 + 2 k \leq 2c \quad \text{It is true} \end{aligned}$$

This relation is true when $n= 2, 3, 4 \dots, k$

$T(k) \leq c k \log k$, where $2 \leq k \leq n$

$T(n/2) \leq c (n/2) \log (n/2)$ ————— (1)

The recurrence relation is

$$T(n) = 2 T(n/2) + k n$$

Apply equation number 1

$$T(n) \leq 2 c (n/2) \log (n/2) + k n$$

$$\leq c n \log (n/2) + k n$$

$$\leq c n \log n - c n \log 2 + k n$$

$$\leq c n \log n - c n + k n$$

$$\leq c n \log n$$

Therefore **$T(n) = O(n \log n)$**

- **Arithmetic Progression**

- Series: $a, a+d, a+2d, a+3d, \dots, a+(n-1)d$
- n^{th} term: $a+(n-1)d$
- sum of first n terms:

$$S_n = \frac{n}{2} [2a + (n-1)d] \quad \text{or} \quad S_n = \frac{n}{2} [T_1 + T_n] \quad \text{or} \quad S_n = n \times (\text{middle term}).$$

- **Geometric progression**

- Series: $a, ar, ar^2, ar^3, \dots, ar^{n-1}$
- n^{th} term: ar^{n-1}
- sum of first n terms:

$$\sum_{k=0}^{n-1} (ar^k) = a \left(\frac{1-r^n}{1-r} \right)$$

- sum of infinite terms:

$$\sum_{k=0}^{\infty} (ar^k) = a \left(\frac{1}{1-r} \right)$$

- **Logarithm related equations**

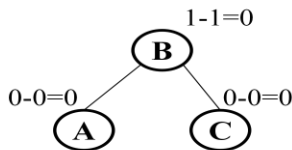
- $x^{\log_a b} = b^{\log_a x}$
- $\log_a a = 1$
- if $x^k = n$, then $k = \log_k n$

- **AVL Trees**

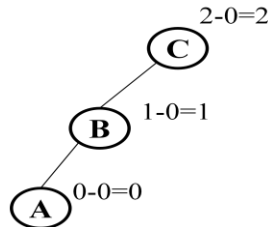
- AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.
- AVL Tree can be defined as **height balanced binary search tree** in which each node is associated with a balance factor.

- **Balance Factor**

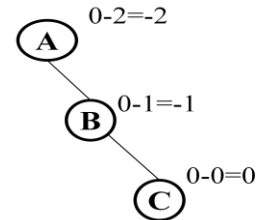
- Balance Factor of a node = height of left subtree – height of right subtree
- In an AVL tree balance factor of every node is -1,0 or +1
- Otherwise the tree will be unbalanced and need to be balanced.



Tree is Balanced.



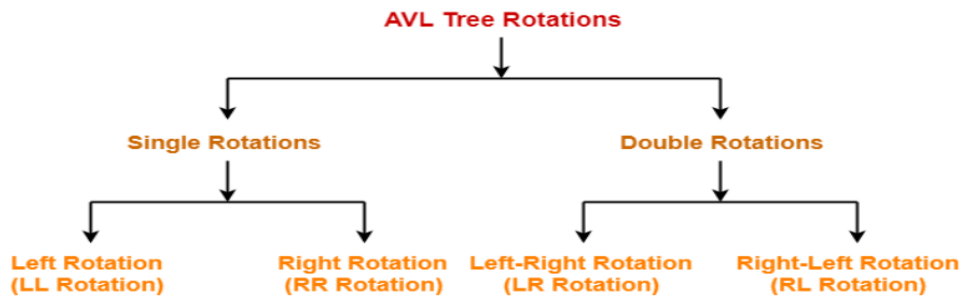
Tree is not Balanced.
Balance factor of C is 2

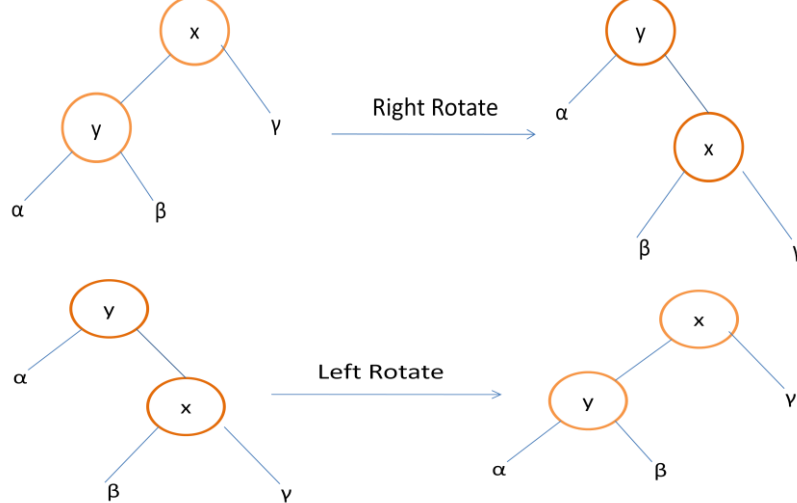


Tree is not Balanced.
Balance factor of A is -2

- **Why AVL Tree?**

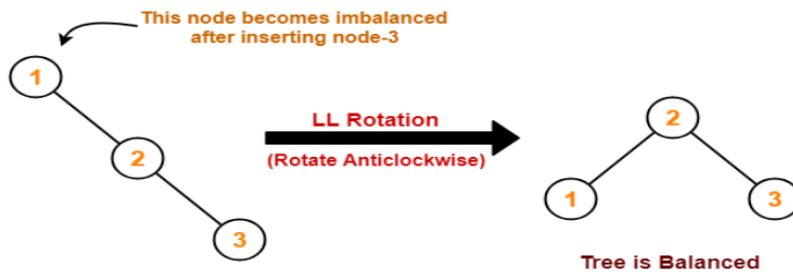
- Most of the Binary Search Tree(BST) operations (eg: search, insertion, deletion etc) take $O(h)$ time where h is the height of the BST.
- The minimum height of the BST is $\log n$
- The height of an AVL tree is always $O(\log n)$ where n is the number of nodes in the tree.
- So the time complexity of all AVL tree operations are $O(\log n)$
- An AVL tree becomes imbalanced due to some insertion or deletion operations
- We use rotation operation to make the tree balanced.
- There are 4 types of rotations





- **Single Left Rotation(LL Rotation)**

- In LL rotation every node moves one position to left from the current position

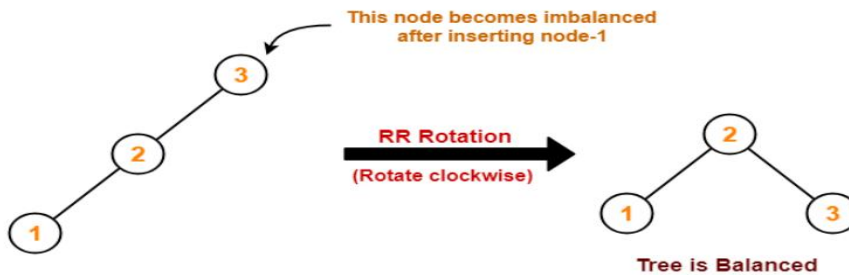


Insertion Order : 1 , 2 , 3

Tree is Imbalanced

- **Single Right Rotation(RR Rotation)**

- In RR rotation every node moves one position to right from the current position

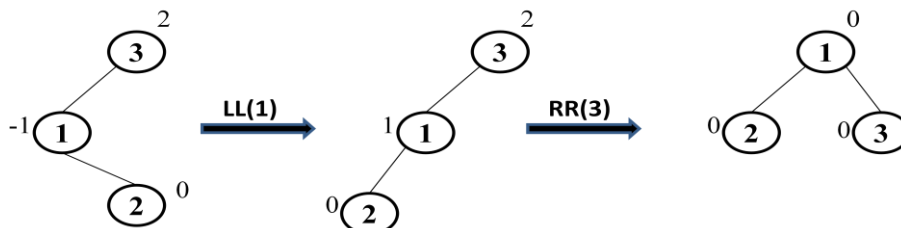


Insertion Order : 3 , 2 , 1

Tree is Imbalanced

- **Left-Right Rotation(LR Rotation)**

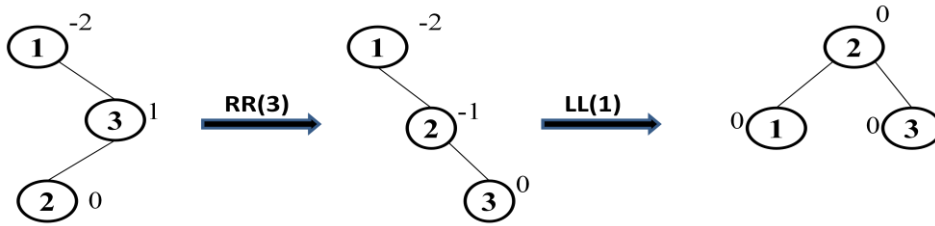
- The LR rotation is the combination of single left rotation followed by single right rotation.



Insertion Order: 3, 1, 2

- **Right-Left Rotation(RL Rotation)**

- The RL rotation is the combination of single right rotation followed by single left rotation.



Insertion Order: 1,3,2

- **AVL Tree Insertion Algorithm**

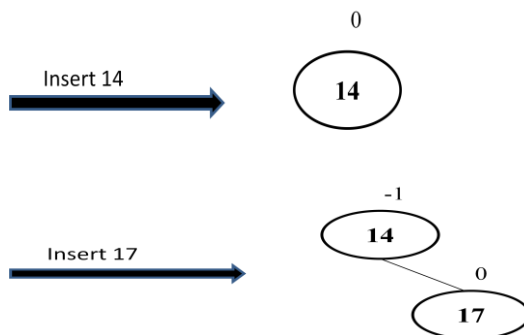
1. Insert the node as the leaf node. Use BST insertion procedure
2. After insertion check the balance factor of every node
3. If the tree is imbalanced, perform the suitable rotation. Let z be the newly inserted node. x be the first unbalanced node on the path from z to root. y be the child of x on the path from z to root
 - If y is the left child of x and z is in the left subtree of y, then perform RR Rotation with respect to x.
 - If y is the right child of x and z is in the right subtree of y, then perform LL Rotation with respect to x.
 - If y is the left child of x and z is in the right subtree of y, then perform LR Rotation
 - If y is the right child of x and z is in the left subtree of y, then perform RL Rotation

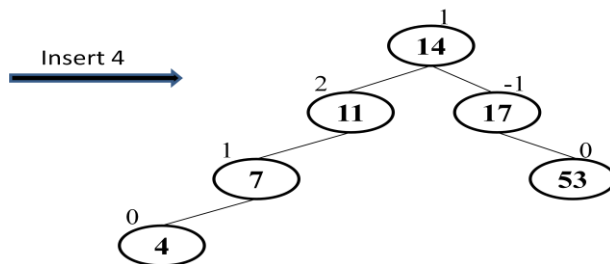
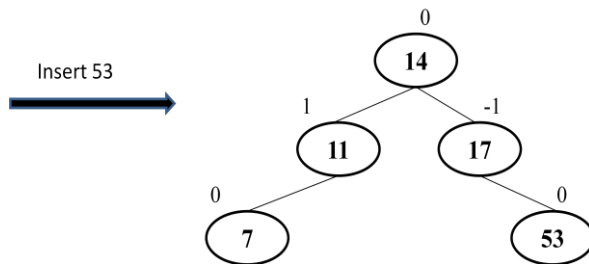
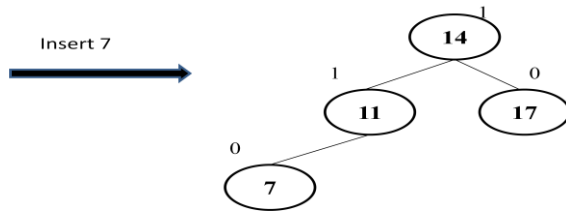
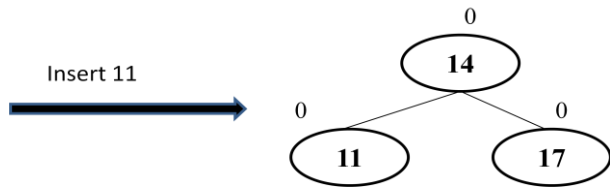
- Complexity of AVL tree insertion = **$O(\log n)$**
Where $\log n$ is the height of the tree.

- **Examples:**

1. Insert 14,17,11,7,53,4 and 13 in to an empty AVL tree

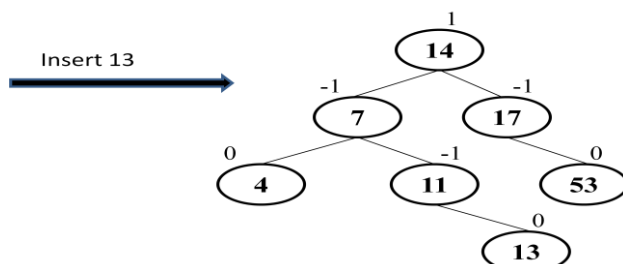
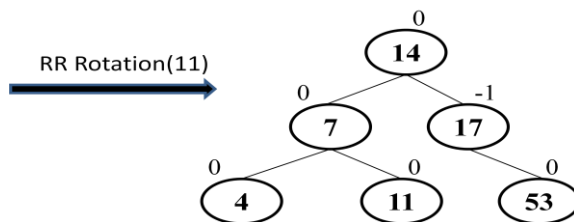
Answer:





The tree is now imbalanced because the balance factor of node 11 is 2.

Perform RR Rotation with respect to 11



2. Insert numbers from 1 to 8 into an empty AVL tree
3. Insert 10,20,15,25,30,16,18 and 19 in to an empty AVL tree

- **AVL Tree Deletion Algorithm**

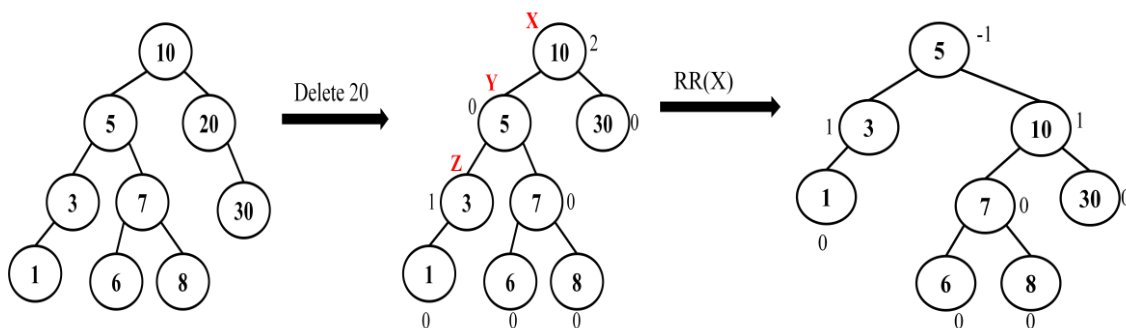
Let w be the node to be deleted

1. Delete w using BST deletion procedure
2. Starting from w , travel up and find the first unbalanced node. Let x be the first unbalanced node.
3. If $\text{balance factor}(x) > 1$ then $y = \text{leftchild}(x)$
4. Else $y = \text{rightchild}(x)$
5. If y is the leftchild of x
 1. If $\text{balance factor}(y) \geq 0$ then $z = \text{leftchild}(y)$
 2. Else $z = \text{rightchild}(y)$
6. Else
 1. If $\text{balance factor}(y) \leq 0$ then $z = \text{rightchild}(y)$
 2. Else $z = \text{leftchild}(y)$
7. If y is the left child of x and z is the left child of y , then perform **RR Rotation** with respect to x
8. If y is the right child of x and z is the right child of y , then perform **LL Rotation** with respect to x
9. If y is the left child of x and z is the right child of y , then perform **LR Rotation** with respect to x
10. If y is the right child of x and z is the left child of y , then perform **RL Rotation** with respect to x

- Complexity of AVL tree deletion = **$O(\log n)$**
Where $\log n$ is the height of the tree.

- **Example**

1. Delete 20 from the given AVL Tree

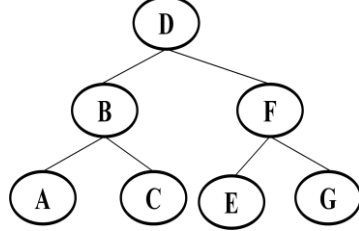


- **University Questions**

1. What is meant by height balanced tree? Give examples
2. Explain the advantages of using height Balanced Trees? Explain AVL Rotations
3. Explain AVL rotations with examples
4. Illustrate the advantage of height balanced binary search trees over binary search trees? Explain various rotations in AVL trees with example.
5. Find minimum and maximum height of any AVL tree with 7 nodes? Assume that the height of a tree with a single node is 0.

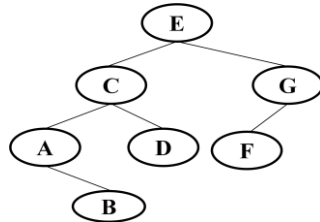
Answer:

Example of minimum height AVL Tree with 7 nodes



Height = 2

Example of maximum height AVL Tree with 7 nodes

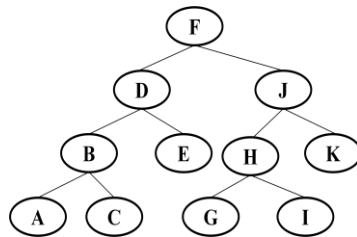


Height = 3

6. Find the minimum and maximum height of any AVL-tree with 11 nodes. Assume that height of the root is 0.

Answer:

Example of minimum and maximum height AVL Tree with 11 nodes



Height = 3

7. Construct AVL tree with the following nodes: 50, 20, 60, 10, 8, 15, 32, 46, 11, 48
8. Define AVL tree. Construct an AVL tree by inserting the keys: 44, 17, 32, 78, 50, 88, 48, 62, 54 into an initially empty tree. Write clearly the type of rotation performed at the time of each insertion.
9. Perform the following operations in the given AVL tree
- Insert 70
 - Delete 55

